STEM Agent: A Self-Adapting, Tool-Enabled, Extensible Architecture for Multi-Protocol AI Agent Systems

Anonymous Authors[1]

Abstract

Current AI agent frameworks suffer from architectural rigidity, single-protocol designs, and an inability to adapt to individual users over time, limiting their deployment across diverse interaction paradigms. We introduce STEM Agent (Self-adapting, Tool-enabled, Extensible, Multi-agent), a modular architecture inspired by the biological stem cell: just as an undifferentiated stem cell can specialize into any tissue type and compose into organs that sustain a living body, the STEM agent core differentiates into specialized protocol handlers, tool bindings, and memory subsystems that compose into a functioning AI system serving business workflows. The framework contributes five interoperability protocols (A2A, AG-UI, A2UI, UCP, AP2) unified behind a single gateway, a Caller Profiler that continuously learns user preferences across 20+ dimensions via exponential moving averages, and an MCP-native design with zero hardcoded domain logic. The implementation comprises 154+ Zod-validated type schemas, 10 self-tunable behavior parameters, and 397 tests passing in under 3 seconds, demonstrating that protocol plurality and behavioral self-adaptation can coexist in a single, extensible agent architecture.

1. Introduction

In developmental biology, a stem cell is remarkable for its pluripotency: it is undifferentiated yet capable of specializing into any cell type—neurons, cardiomyocytes, hepatocytes—which then compose into organs that sustain a living body. We take this as a design principle for AI agent systems. The STEM Agent (Self-adapting, Tool-enabled, Extensible, Multi-agent) is an undifferentiated agent core that differentiates into specialized protocol handlers, tool bindings, and memory types, which compose into a functioning system that supports business projects—much as organs compose into a body supporting life.

This analogy is not merely rhetorical. Current agent frameworks exhibit a form of "terminal differentiation": they commit early to a single interaction protocol (e.g., REST-only or chat-only), a fixed tool integration strategy, and static user models. The result is an ecosystem of rigid, single-purpose agents that cannot interoperate, adapt, or compose (Ferrag et al., 2025; Tran et al., 2025). Meanwhile, the proliferation of agent communication protocols—MCP (Anthropic, 2024), A2A (Habler et al., 2025), and emerging standards for UI streaming and commerce—demands architectures that are protocol-pluralistic by design (Ehtesham et al., 2025).

STEM Agent addresses these gaps with the following contributions:

1. Five-protocol interoperability. The first agent framework implementing A2A (agent-to-agent), AG-UI (streaming UI events), A2UI (dynamic UI composition), UCP (universal commerce), and AP2 (agent payments) behind a unified Express.js gateway.

2. Caller Profiler. A multi-dimensional user modeling system that continuously learns caller preferences across 4 categories and 20+ dimensions using exponential moving averages, enabling per-user behavioral adaptation.

3. MCP-native design. All external capabilities are acquired at runtime via the Model Context Protocol, yielding zero hardcoded domain logic and full separation of agent reasoning from domain knowledge.

4. Self-tunable behavior parameters. Ten continuously adjusted parameters (reasoning depth, cre-

[1]AUTHORERR: Missing \icmlaffiliation. .AUTHORERR: Missing \icmlcorrespondingauthor.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

ativity, verbosity, etc.) that adapt to task characteristics and caller profiles without manual tuning.

5. Commerce-ready agent protocols. Novel UCP and AP2 protocol handlers enabling checkout sessions, mandate-based payments, and audit trails within the agent interaction loop.

The remainder of the paper is organized as follows. Section 2 surveys related work. Section 3 presents the five-layer architecture and cognitive pipeline. Section 4 details multi-protocol interoperability. Section 5 describes self-adaptation and learning mechanisms. Section 6 reports implementation metrics and framework comparisons. Section 7 concludes with future directions.

2. Related Work

Multi-agent frameworks. AutoGen (Wu et al., 2023) introduced flexible multi-agent conversation with human-in-the-loop control. MetaGPT (Hong et al., 2023) assigns structured roles (PM, architect, engineer) to agents in software development workflows. CAMEL (Li et al., 2023) uses inception prompting for role-playing task decomposition. CrewAI and LangGraph (Duan & Wang, 2024) provide graph-based orchestration with state management. These frameworks advance multi-agent coordination but typically commit to a single communication protocol and lack per-user adaptation. Orogat et al. (2026) provide a recent benchmark comparing these systems. STEM Agent extends this line of work by supporting five protocols simultaneously and adding continuous caller modeling.

Agent communication protocols. The Model Context Protocol (MCP) (Anthropic, 2024) standardizes tool and resource access for LLM applications. The Agent-to-Agent protocol (A2A) (Habler et al., 2025) enables inter-agent communication via JSON-RPC 2.0. Ehtesham et al. (2025) survey MCP, A2A, ACP, and ANP, identifying interoperability gaps. Li & Xie (2025) analyze the integration challenges of combining A2A and MCP. The Agent Network Protocol (ANP) (Chang et al., 2025) addresses decentralized agent discovery. Despite this proliferation, no existing framework implements more than two of these protocols. STEM Agent implements five, including novel commerce protocols (UCP, AP2).

Reasoning strategies. Chain-of-Thought prompting (Wei et al., 2022) established step-by-step reasoning for LLMs. ReAct (Yao et al., 2023b) interleaves reasoning with tool actions. Reflexion (Shinn et al., 2023) adds verbal self-reflection. Tree-of-Thought (Yao et al., 2023a) explores multiple reasoning paths. Wei et al. (2026) survey recent advances in agentic reasoning. STEM Agent integrates four reasoning strategies (Chain-of-Thought, ReAct, Reflexion, Internal Debate) with automatic selection based on task characteristics.

Tool-augmented LLMs. Toolformer (Schick et al., 2023) demonstrated self-taught tool use. Gorilla (Patil et al., 2023) fine-tunes LLMs for API calls. ToolLLM (Qin et al., 2023) scales to 16,000+ real-world APIs. These approaches embed tool knowledge in model weights. STEM Agent instead externalizes all tool access through MCP, enabling dynamic capability discovery without retraining.

Self-adaptive systems. Liu et al. (2022) formalize self-initiated continual learning for open-world AI autonomy. Gheibi & Weyns (2022) combine lifelong machine learning with self-adaptive software systems. STEM Agent operationalizes these principles through its Caller Profiler and self-tunable behavior parameters, achieving continuous adaptation within a deployed agent system.

Table 1. Five-layer architecture overview.

Layer	Component	Key Counts
1	Caller / User Layer	4 adapters
2	Standard Interface (Gateway)	5 protocols
3	Agent Core	4 engines
3.5	Security Layer	4 IAM plugins
4	Memory System	4 memory types
5	MCP Integration	3 transports

3. Architecture

STEM Agent is organized into five layers, each corresponding to a distinct concern (Table 1). The design follows the stem cell analogy: Layer 3 (Agent Core) is the undifferentiated "stem cell" that differentiates through Layer 2 (protocol handlers) and Layer 5 (tool bindings) into specialized capabilities, while Layer 4 (Memory) provides the "epigenetic" state that guides future differentiation.

3.1. Cognitive Pipeline

The Agent Core processes each request through a seven-phase cognitive pipeline (Algorithm 1). The pipeline is inspired by cognitive science models of hu-

Algorithm 1 STEM Cognitive Pipeline

Input: message m, caller context c
Output: response r, updated profile c'

// Phase 1: Perception
$p \leftarrow \text{Perceive}(m)$ ▷ intent, entities, complexity

// Phase 2: Adaptation
$a \leftarrow \text{Adapt}(c, p)$ ▷ load profile, adjust params

// Phase 3: Reasoning
$s \leftarrow \text{SelectStrategy}(p, a)$
$R \leftarrow \text{Reason}(m, p, a, s)$ ▷ multi-step reasoning

// Phase 4: Planning
$P \leftarrow \text{Plan}(R, \text{McpTools}())$ ▷ tool selection

// Phase 5: Execution
$E \leftarrow \text{Execute}(P)$ ▷ MCP tool calls

// Phase 6: Formatting
$r \leftarrow \text{Format}(E, a)$ ▷ style for caller

// Phase 7: Learning (async)
$c' \leftarrow \text{UpdateProfile}(c, p)$
ExtractKnowledge(m, r)
return r, c'

man information processing (Park et al., 2023) but adapted for the constraints of LLM-based agents.

The Perceive phase classifies intent into 10 categories, estimates complexity on a $[0, 1]$ scale, extracts entities, and detects sentiment and urgency. The Adapt phase loads the caller's learned profile and adjusts behavior parameters accordingly. SelectStrategy chooses among four reasoning strategies based on task characteristics: tool-requiring tasks use ReAct, complexity > 0.8 triggers Reflexion, analysis and creative requests use Internal Debate, and all others default to Chain-of-Thought. The Plan phase selects MCP tools and constructs an execution plan with parallel steps where dependencies allow. Execute orchestrates MCP tool calls with retries (default: 2) and a circuit breaker (threshold: 3 consecutive failures). The Learning phase runs asynchronously, updating the caller profile and extracting knowledge triples for the semantic memory.

3.2. Implementation

The system is implemented as a TypeScript monorepo with six workspace packages: shared (154+ Zod schemas), agent-core (cognitive engines), standard-interface (protocol handlers and gateway), mcp-integration (MCP client layer), memory-system (four memory stores), and caller-layer (caller utilities). All inter-layer communication uses Zod-validated schemas, providing runtime type safety across the entire pipeline. The gateway is built on Express.js 5, with each protocol handler mounted via a pluggable createRouter() pattern.

4. Multi-Protocol Interoperability

A key contribution of STEM Agent is the simultaneous support for five interaction protocols behind a unified gateway. Table 2 summarizes each protocol.

A2A (Agent-to-Agent). Implements the A2A v0.3.0 specification (Linux Foundation) using JSON-RPC 2.0. Supports tasks/send, tasks/sendSubscribe (SSE streaming), tasks/get, and tasks/cancel. Agent discovery is served at /.well-known/agent.json.

AG-UI (Agent-User Interaction). Streams cognitive pipeline events to frontends via Server-Sent Events. Events include TEXT_MESSAGE_START, REASONING_MESSAGE, TOOL_CALL_START, STATE_SNAPSHOT, and RUN_FINISHED, enabling fine-grained progress rendering.

A2UI (Agent-to-User Interface). Provides dynamic UI composition through 16 component primitives (text, button, card, list, text field, checkbox, radio, select, slider, toggle, image, divider, spacer, progress bar, alert, badge). Components are organized in a flat adjacency list model where each component references children by ID, enabling flexible layout construction.

UCP (Universal Commerce Protocol). Manages checkout session lifecycles with required idempotency headers (Idempotency-Key, Request-Id, UCP-Agent). Sessions progress through creation, retrieval, and completion endpoints. An idempotency cache prevents duplicate session creation.

AP2 (Agent Payments Protocol). Implements a three-phase payment lifecycle: Intent Mandate → Payment Mandate → Payment Receipt. Supports auto-approval for payments below a configurable threshold and maintains a full audit trail per intent.

Gateway architecture. Each protocol handler extends a common pattern: a handler class implements a createRouter(): Router method that returns an Express.js router. The gateway's setupRoutes() method mounts all routers, along with shared middleware for

Table 2. Protocol comparison. STEM Agent implements all five behind a single Express.js gateway.

Protocol	Endpoint	Transport	Schemas	Use Case
A2A	POST /a2a	JSON-RPC 2.0	6	Agent-to-agent task delegation, inter-agent messaging, agent card discovery
AG-UI	POST /ag-ui	SSE	20	Real-time UI streaming; maps pipeline phases to typed events
A2UI	POST /a2ui/render	SSE + REST	19	Dynamic UI composition with 16 component primitives
UCP	POST /ucp/*	REST	14	Commerce checkout sessions with idempotency
AP2	POST /ap2/*	REST	10	Mandate-based payments with audit trail

Table 3. Caller Profile dimensions by category.

Category	Dims	Example Dimensions
Philosophy	8	pragmatism vs. idealism, risk tolerance, innovation orientation
Principles	4	correctness over speed, testing emphasis, security mindedness
Style	5	formality, verbosity, technical depth, structure preference
Habits	4+	session length, iteration tendency, peak hours

authentication, rate limiting, request correlation, and error handling. This pattern makes adding a new protocol a localized change: implement the handler, mount the router, export from the package index.

Framework adapters. Four adapters (AutoGen, CrewAI, LangGraph, OpenAI Agents SDK) translate between external framework conventions and STEM Agent's internal message format, enabling integration with existing multi-agent ecosystems.

5. Self-Adaptation and Learning

STEM Agent's self-adaptation operates at two levels: per-caller profile learning and per-task behavior parameter tuning.

5.1. Caller Profiler

The Caller Profiler learns a multi-dimensional model of each user across four categories (Table 3): philosophy (8 dimensions, e.g., pragmatism vs. idealism, risk tolerance), principles (4 dimensions, e.g., correctness over speed, testing emphasis), style (5 dimensions, e.g., formality, verbosity, technical depth), and habits (temporal and behavioral patterns).

Learning mechanism. Each dimension is updated via an exponential moving average (EMA) with learning rate $\alpha = 0.1$:

$$v_{t+1} = (1 - \alpha) \cdot v_t + \alpha \cdot s_t \tag{1}$$

where v_t is the current profile value and s_t is the signal extracted from the current interaction. Signals are extracted by the Perception Engine, which analyzes each message for philosophy, principles, style, and habit indicators.

Confidence-based adaptation. Profile confidence follows a sigmoid ramp:

$$\text{conf}(n) = 1 - e^{-0.1 \cdot n} \tag{2}$$

where n is the number of interactions. A minimum of 5 interactions is required before the profile influences behavior ($\text{conf}(5) \approx 0.39$), reaching ≈ 0.86 at 20 interactions. Below the confidence threshold, the system relies on signals from the current message; above it, the learned profile is blended with current signals weighted by confidence.

5.2. Behavior Parameters

Ten parameters are continuously adjusted based on task characteristics and caller profile:

- Reasoning depth (default: 3): Controls multi-step reasoning iterations.
- Exploration vs. exploitation (default: 0.3): Balances novel vs. proven strategies.
- Verbosity level (default: 0.5): Response detail.
- Confidence threshold (default: 0.7): Minimum confidence to act.
- Tool use preference (default: 0.5): Tendency to invoke tools.

- Creativity level (default: 0.5): Solution novelty.
- Proactive suggestion (default: on): Unsolicited recommendations.
- Self-reflection frequency (default: every 5 steps).
- Max plan steps (default: 10).
- Memory retrieval breadth (default: 10).

These parameters are adjusted by the Adaptation phase of the cognitive pipeline, using the caller profile and current task perception as inputs.

5.3. Memory System

The memory system implements four types inspired by cognitive science (Lewis et al., 2020; Park et al., 2023):

1. Episodic memory: Stores specific interaction episodes with vector embeddings for similarity search (PostgreSQL + pgvector). Each episode carries an importance score.
2. Semantic memory: Maintains knowledge triples (subject, predicate, object) in concept graphs, with pattern extraction from episodic episodes.
3. Procedural memory: Records successful strategies and tool usage patterns, enabling best-procedure matching for recurring task types.
4. User context memory: Per-caller session history and profiles with GDPR forget-me support.

A Memory Manager provides a unified facade, delegating to specialized modules and performing memory consolidation (episodic → semantic/procedural) as interaction history grows.

5.4. Reasoning Strategy Selection

The Strategy Selector maps task characteristics to reasoning strategies:

- Tool-requiring tasks → ReAct (Yao et al., 2023b)
- Complexity > 0.8 → Reflexion (Shinn et al., 2023)
- Analysis or creative requests → Internal Debate (Du et al., 2023)
- Default → Chain-of-Thought (Wei et al., 2022)

This automatic selection eliminates the need for users or developers to manually specify reasoning approaches, contributing to the system's self-adaptive character.

Table 4. Comparison with existing agent frameworks.

Framework	Proto.	Adapt	Mem.	Com.	Tests
AutoGen	1	×	1	×	–
MetaGPT	1	×	1	×	–
CrewAI	1	×	1	×	–
LangChain	1	×	2	×	–
STEM	5	✓	4	✓	397

Proto. = interoperability protocols; Adapt = self-adaptive behavior; Mem. = memory types; Com. = commerce protocols.

6. Implementation and Evaluation

6.1. Implementation Details

STEM Agent is implemented as a TypeScript monorepo comprising six workspace packages. Key dependencies include the Anthropic SDK for LLM integration, Zod for runtime type validation (154+ exported schemas), Express.js 5 for the HTTP gateway, Pino for structured JSON logging, and the MCP SDK for protocol compliance. The MCP integration layer supports two transport types (stdio, SSE) and six server wrappers (database, API, file system, tool, custom, base).

The security layer implements a pluggable IAM architecture with four authentication plugins (JWT with RS256, OAuth2 with token introspection, SAML, API Key) behind a SecurityManager with a TTL-based policy cache (1000 entries, 5-minute TTL). Rate limiting uses a token bucket algorithm (60 requests/minute, burst 10).

6.2. Test Suite

The test suite comprises 397 tests across 36 test files using Vitest, with 100% pass rate and a total runtime of 2.86 seconds. Tests cover unit tests per engine (perception, reasoning, planning, execution), protocol handler integration tests, memory system tests, MCP integration tests, security middleware tests, framework adapter tests, and gateway end-to-end tests.

6.3. Framework Comparison

Table 4 compares STEM Agent with existing frameworks across key dimensions.

No existing framework supports more than one standardized interoperability protocol. None provides continuous per-caller adaptation or commerce-ready payment protocols. STEM Agent's MCP-native design further differentiates it by externalizing all domain knowledge through dynamic tool discovery, whereas

other frameworks embed domain logic in code or configuration.

Protocol compliance. Each protocol handler is validated against its specification: A2A against the v0.3.0 Linux Foundation spec, MCP against the 2025-11-25 SDK specification. The AG-UI, A2UI, UCP, and AP2 handlers are validated through comprehensive integration tests that verify event sequences, error handling, and idempotency guarantees.

Security. The pluggable IAM architecture supports enterprise deployment requirements. All authentication and authorization events are audit-logged. The four IAM plugins can be composed (e.g., JWT for agents, API Key for service accounts), and custom plugins can be added without modifying core code.

7. Conclusion and Future Work

We have presented STEM Agent, an architecture that embodies the stem cell principle: an undifferentiated core that differentiates into specialized protocol handlers, tool bindings, and memory subsystems, composing into a functioning AI system that supports diverse business workflows. The framework makes five contributions: (1) the first five-protocol agent gateway, (2) a Caller Profiler with continuous multi-dimensional learning, (3) MCP-native design with zero hardcoded domain logic, (4) self-tunable behavior parameters, and (5) commerce-ready agent protocols.

Several directions remain for future work. First, integrating Tree-of-Thought reasoning (Yao et al., 2023a) would enable multi-path exploration for complex planning tasks. Second, a Swarm collaboration pattern would support parallel solution-space exploration across agent teams. Third, meta-learning (Gheibi & Weyns, 2022) over the distribution of tasks seen by the Caller Profiler could accelerate adaptation for new users by transferring knowledge from similar profiles. Finally, additional protocols such as ANP (Chang et al., 2025) and AWCP (Nie et al., 2026) could further expand interoperability.

Impact Statement

This paper presents work whose goal is to advance the field of AI agent systems. The STEM Agent architecture is designed for transparency (audit logging, explainable adaptation) and user control (GDPR forget-me support, configurable autonomy levels). As with any agent framework, deployment should consider safety boundaries, particularly when commerce protocols handle financial transactions. We encourage adopters to implement appropriate guardrails for their deployment context.

References

Anthropic. Model context protocol specification. Technical report, Anthropic, 2024. https://modelcontextprotocol.io.

Chang, G., Lin, E., Yuan, C., Cai, R., Chen, B., Xie, X., and Zhang, Y. Agent network protocol technical white paper. arXiv preprint arXiv:2508.00007, 2025.

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. arXiv preprint arXiv:2305.14325, 2023.

Duan, Z. and Wang, J. Exploration of LLM multi-agent application implementation based on LangGraph+CrewAI. arXiv preprint arXiv:2411.18241, 2024.

Ehtesham, A., Singh, A., Gupta, G. K., and Kumar, S. A survey of agent interoperability protocols: Model context protocol (MCP), agent communication protocol (ACP), agent-to-agent protocol (A2A), and agent network protocol (ANP). arXiv preprint arXiv:2505.02279, 2025.

Ferrag, M. A., Tihanyi, N., and Debbah, M. From LLM reasoning to autonomous AI agents: A comprehensive review. arXiv preprint arXiv:2504.19678, 2025.

Gheibi, O. and Weyns, D. Lifelong self-adaptation: Self-adaptation meets lifelong machine learning. In Proceedings of the 17th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp. 145–151, 2022. doi: 10.1145/3524844.3528052.

Habler, I., Huang, K., Narajala, V. S., and Kulkarni, P. Building a secure agentic AI application leveraging A2A protocol. arXiv preprint arXiv:2504.16902, 2025.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., et al. MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv preprint arXiv:2308.00352, 2023.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive NLP

tasks. In Advances in Neural Information Processing Systems, volume 33, pp. 9459–9474, 2020.

Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., and Ghanem, B. CAMEL: Communicative agents for "mind" exploration of large language model society. Advances in Neural Information Processing Systems, 36, 2023.

Li, Q. and Xie, Y. From glue-code to protocols: A critical analysis of A2A and MCP integration for scalable agent systems. arXiv preprint arXiv:2505.03864, 2025.

Liu, B., Mazumder, S., Robertson, E., and Grigsby, S. AI autonomy: Self-initiated open-world continual learning and adaptation. arXiv preprint arXiv:2203.08994, 2022.

Nie, X., Guo, Z., Chen, Y., Zhou, Y., and Zhang, W. AWCP: A workspace delegation protocol for deep-engagement collaboration across remote agents. arXiv preprint arXiv:2602.20493, 2026.

Orogat, A., Rostam, A., and Mansour, E. Understanding multi-agent LLM frameworks: A unified benchmark and experimental analysis. arXiv preprint arXiv:2602.03128, 2026.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, pp. 1–22, 2023.

Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: Large language model connected with massive APIs. arXiv preprint arXiv:2305.15334, 2023.

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. arXiv preprint arXiv:2307.16789, 2023.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems, volume 36, 2023.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, volume 36, 2023.

Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., and Nguyen, H. D. Multi-agent collaboration mechanisms: A survey of LLMs. arXiv preprint arXiv:2501.06322, 2025.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems, volume 35, pp. 24824–24837, 2022.

Wei, T., Li, T.-W., Liu, Z., Ning, X., Yang, Z., Zou, J., Zeng, Z., Qiu, R., Lin, X., Fu, D., et al. Agentic reasoning for large language models. arXiv preprint arXiv:2601.12538, 2026.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. Advances in Neural Information Processing Systems, 36, 2023a.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023b.